

Національний університет «Одеська Політехніка»

Інститут комп'ютерних систем

Кафедра інформаційних систем

Лабораторна робота №4

з дисципліни «Програмування мобільних пристроїв на базі ОС Android»

Тема: «Локальна база даних Room та Repository в Android-застосунку»

Виконав:

студент групи AI-234

Середа Я. В.

Перевірили:

Годовиченко М. А.

Соловйова Д. В.

Одеса 2026

МЕТА РОБОТИ

Ознайомитися з принципами локального зберігання даних в Android-застосунках за допомогою бібліотеки Room. Сформуванати навички побудови багаторівневої структури застосунку з розділенням відповідальності між UI, ViewModel, Repository та локальною базою даних. Опанувати перехід від зберігання даних у пам'яті до персистентного зберігання в SQLite.

ЗАВДАННЯ

- Відкрити проєкт багатоекранного застосунку «Мої завдання» (Варіант 4), створений у лабораторній роботі 3.
- Додати до проєкту залежності бібліотеки Room та плагін KSP.
- Створити Entity-клас для сутності завдання.
- Створити DAO-інтерфейс для виконання CRUD-операцій (додавання, отримання, оновлення, видалення).
- Створити клас бази даних `AppDatabase`, що наслідує `RoomDatabase`.
- Реалізувати патерн Repository як окремий рівень абстракції для роботи з даними.
- Переписати `TasksViewModel` так, щоб вона отримувала дані реактивно через Repository з локальної бази даних.
- Перевірити персистентність даних (збереження після закриття та повторного запуску застосунку).

1 ХІД РОБОТИ

1.1 Налаштування середовища та залежностей

У файл `build.gradle.kts` модуля `app` було додано плагін KSP та залежності бібліотеки Room.

1.2 Створення Entity-класу та маппера

Створено клас `TaskEntity` з анотацією `@Entity(tableName = "tasks")`. Поле `id` налаштовано як `@PrimaryKey(autoGenerate = true)`. Для забезпечення чистоти архітектури розроблено `TaskMapper` у вигляді функцій-розширень, які конвертують об'єкти бази даних у доменні моделі UI і навпаки.

1.3 Розробка DAO та бази даних

Створено інтерфейс `TaskDao` з анотацією `@Dao`. Реалізовано SQL-запити для отримання списку завдань та окремого завдання за ID. Додано функції для `@Insert`, `@Update` та `@Delete`.

Створено абстрактний клас `AppDatabase`, який ініціалізується як Singleton за допомогою `Room.databaseBuilder`.

1.4 Реалізація Repository

Створено клас `TaskRepository`, який інкапсулює логіку доступу до `TaskDao`. Репозиторій отримує реактивний потік `Flow<List<TaskEntity>>` від DAO, мапить його у `Flow<List<TaskItem>>` та віддає на рівень ViewModel. Це забезпечило розв'язання бізнес-логіки та логіки зберігання даних.

1.5 Переписування ViewModel

Клас `TasksViewModel` було повністю оновлено. Вилучено локальну колекцію `MutableStateFlow` зі статичними даними. Замість неї змінна `tasksListUiState` тепер ініціалізується шляхом підписки на потік даних з `TaskRepository` за допомогою оператора `stateIn`.

Всі операції зміни даних тепер виконуються у `viewModelScope.launch` і викликають відповідні методи репозиторію.

Для ініціалізації ViewModel із передачею Repository в конструктор створено `TasksViewModelFactory`.

2 ПЕРЕВІРКА ФУНКЦІОНАЛЬНОСТІ ДОДАТКУ

2.1 Перевірка створення бази даних

Після першого запуску додатка було використано інструмент Database Inspector в Android Studio. Перевірка підтвердила, що база даних `tasks_database` успішно створена, а таблиця `tasks` має правильну структуру з визначеними полями (рис. 2.1).

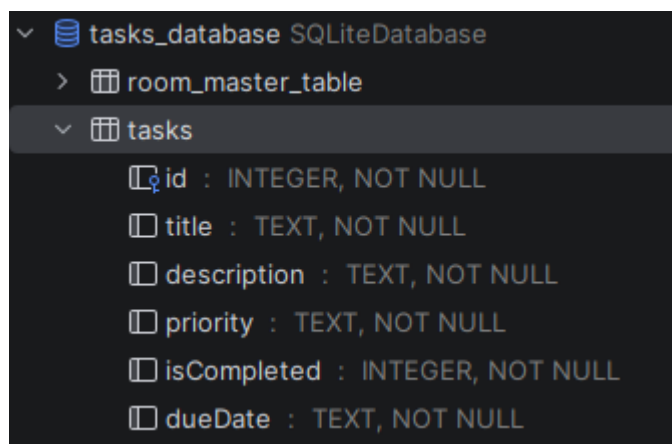


Рисунок 2.1 – Структура бази даних у Database Inspector

2.2 Додавання та зчитування даних

Додане нове завдання “test” через форму створення (рис. 2.2). Після збереження UI миттєво оновився завдяки реактивному потоку `Flow` від бази даних. У Database Inspector видно, що запис успішно додано до таблиці (рис. 2.3).

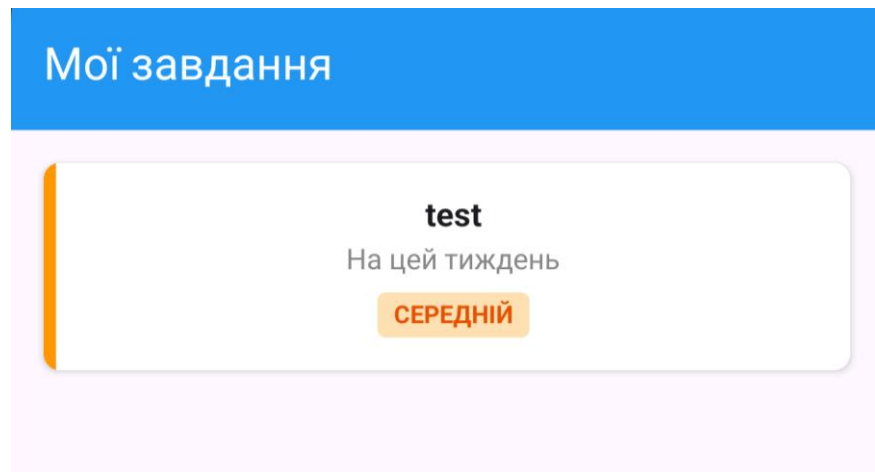


Рисунок 2.2 – Відображення нового завдання у списку на головному екрані

	id	title	description	priority	isCompleted	dueDate
1	2	test	123	MEDIUM	0	На цей тиждень

Рисунок 2.3 – Відповідний запис про нове завдання у БД

2.3 Оновлення статусу та редагування

Через екран деталей було змінено статус завдання на «Виконане». Інтерфейс успішно відреагував (рис. 2.4), а в базі даних значення поля `isCompleted` автоматично змінилося з 0 на 1 (рис. 2.5).

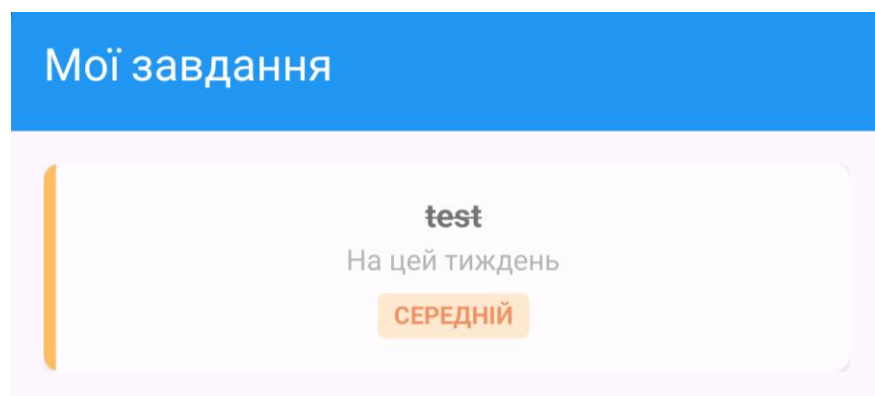


Рисунок 2.4 – Оновлення статусу завдання на головному екрані

	id	title	description	priority	isCompleted	dueDate
1	2	test	123	MEDIUM	1	На цей тиждень

Рисунок 2.5 – Зміна значення **isCompleted** на 1 у БД

2.4 Перевірка персистентності даних

Застосунок було повністю закрито (рис. 2.6). Після повторного запуску (рис. 2.7) завдання залишилося на головному екрані зі збереженим статусом «Виконане», що доводить успішне завантаження даних із локального сховища SQLite.

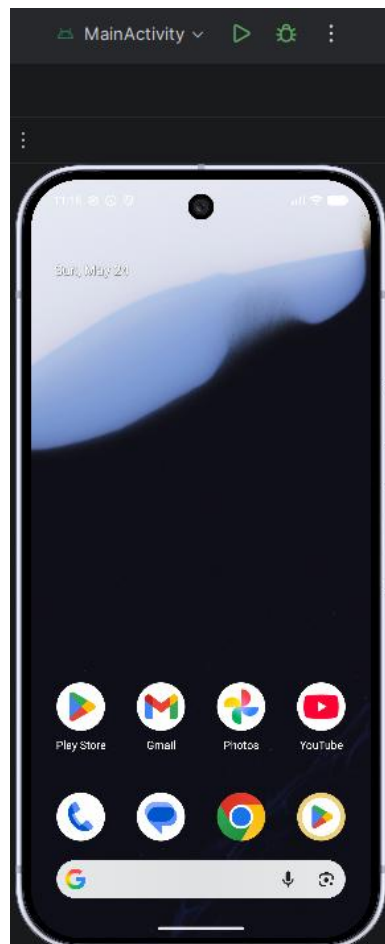


Рисунок 2.6 – Повне закриття застосунку

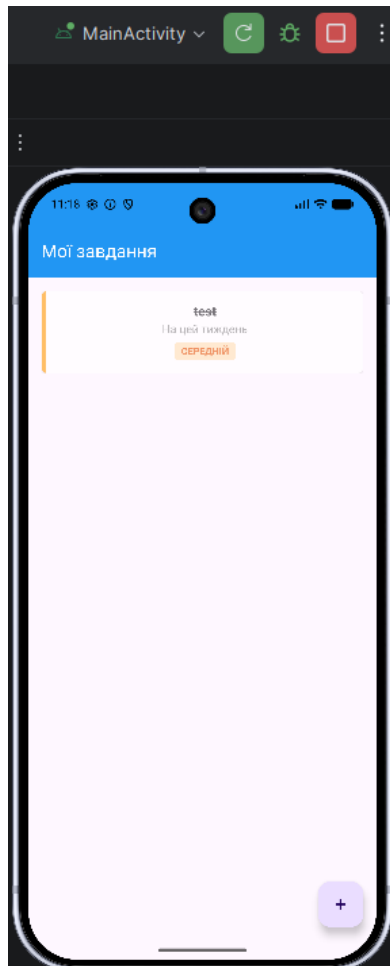


Рисунок 2.7 – Стан списку завдань після перезапуску

2.5 Перевірка видалення даних

Завдання було видалено через відповідну кнопку. Запис успішно зник з екрана (рис. 2.8), а також був фізично видалений з бази даних (рис. 2.9).

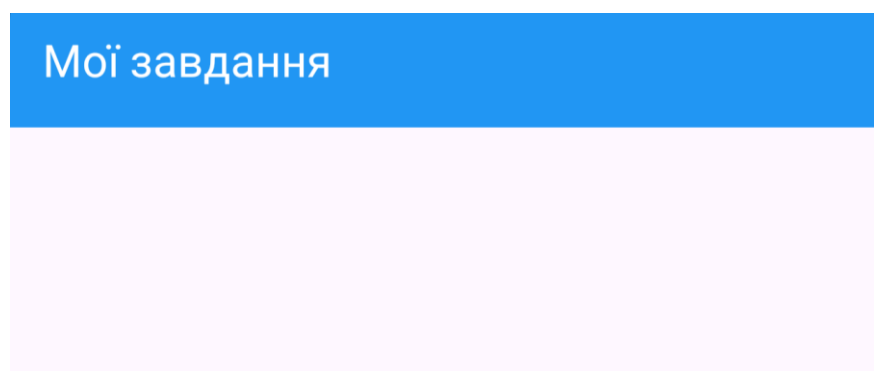
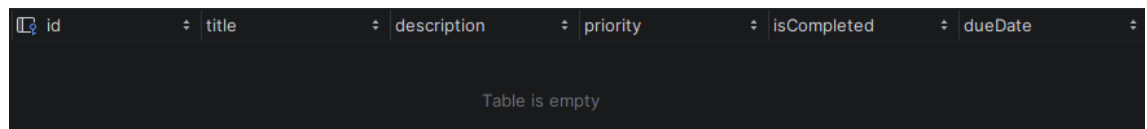


Рисунок 2.8 – Головний екран після видалення завдання



The image shows a dark-themed interface for a database table. At the top, there is a header bar with six column names: 'id', 'title', 'description', 'priority', 'isCompleted', and 'dueDate'. Each column name is preceded by a small icon of a table with a blue arrow pointing to it. Below the header bar, the text 'Table is empty' is centered in a light gray font.

id	title	description	priority	isCompleted	dueDate
Table is empty					

Рисунок 2.9 – Порожня таблиця в Database Inspector

ВИСНОВОК

Під час виконання лабораторної роботи було успішно інтегровано локальну базу даних Room у багатоекранний Android-застосунок, створений за допомогою Jetpack Compose.

Практично закріплено навички роботи з ключовими компонентами Room: Entity, DAO та Database. Успішно реалізовано патерн Repository, що дозволило приховати деталі реалізації локального сховища від ViewModel.

Перехід від зберігання даних в оперативній пам'яті до використання SQLite забезпечив персистентність даних – тепер інформація про завдання не втрачається після закриття застосунку, як це було у попередній роботі. Завдяки використанню Kotlin Coroutines та Flow, отримання та оновлення даних з бази відбувається асинхронно і реактивно, автоматично оновлюючи інтерфейс користувача при будь-яких змінах у базі даних.